

Task-Contract-Aware Online Inverse Kinematics: Aligning Solver Convergence with Command Admissibility

Author information to be supplied for submission

Abstract

Online inverse kinematics is often judged by solver-specific convergence, although a robot needs commands satisfying task-level pose, joint and rate constraints within a finite time. We study this mismatch through a task-contract-aware interface that conservatively maps public pose tolerances into native solver conditions, imposes dynamic joint bounds and independently verifies every return. We compare official TRAC-IK and an existing adaptive damped-least-squares solver on Panda and UR5e using witnessed points, contract-scale sensitivity and complete online trajectories. Strict DLS reports nonconvergence for admissible returns on 21.92% and 21.24% of feasible points. Strict TRAC-IK already solves all nominal witnessed points; full alignment preserves that success with modest computational savings. On continuous trajectories, aligned TRAC-IK at a 5 ms search budget increases completion from 34 to 36–37 of 40 Panda paths across repeated runs, and from 37 to 39 on UR5e, reducing cumulative time by 32.53% and 42.59%. DLS alignment reduces computation but loses UR5e trajectory completions, exposing a closed-loop trade-off. A frozen learned-allocation case study likewise finds no universal acceleration of successful solving. The evidence supports aligning solver conditions with the task contract before adding adaptive computation, while evaluating complete tracking rather than local acceptance alone.

Keywords: inverse kinematics; task tolerance; command admissibility; joint-rate constraints; TRAC-IK; damped least squares; algorithm selection

1 Introduction

A robot receiving Cartesian targets from a planner, a teleoperator or a learned policy must convert them into joint commands that can be issued from its current configuration. This repeated use of inverse kinematics (IK) differs from solving an isolated geometric equation. The input includes both the desired end-effector pose and the last accepted joint state, while the time available to return the next command is finite. A numerically accurate solution can be unsuitable if reaching it requires excessive joint motion during the command interval. Conversely, an approximate solution may already meet every requirement of the task. Online IK is therefore a command-generation problem: its useful output is a configuration that satisfies specified execution conditions at an acceptable computational cost. Continuity matters because each accepted configuration determines the starting point of the next query [1, 2]. Evaluating this process requires more than counting roots or recording a library’s success flag. It requires an explicit account of what the robot is allowed to accept, how a numerical return is checked, and whether the complete interface returns that command before its deadline. The relevant latency includes preparing solver inputs and checking the result, since neither operation can be omitted by an application issuing the command.

Analytical and numerical IK offer strong foundations for this purpose. Analytical solutions exploit manipulator geometry and can be extremely efficient when a robot admits a suitable decomposition. Recent geometric solvers broaden the range of structures handled systematically and make this option easier to use [3, 4]. Jacobian methods offer a direct local relationship between Cartesian and joint motion; damped least squares (DLS) moderates the amplification associated with small singular values [5–7]. TRAC-IK combines complementary numerical searches to address failures associated with joint limits and local search behavior [8]. Quadratic programming and constrained optimization can represent joint and motion restrictions explicitly, and structure-aware numerical implementations reduce their computational burden [9]. These methods solve different numerical formulations, with different stopping rules and search budgets, but all can serve an online command-generation pipeline. The question considered here concerns the interface around these established solvers. Their geometric and numerical capabilities should be assessed under the same task requirements before differences in internal precision or return semantics are interpreted as differences in useful solving ability.

The task itself usually supplies a set of acceptable outcomes rather than a single infinitely precise mathematical root. A target pose can have separate position and orientation allowances; joint ranges restrict the available configuration space; and the command interval converts velocity limits into per-joint admissible displacements. Set-based tasks and pose-constrained manipulation already express the value of admissible regions [10, 11].

Such task requirements differ from the residual threshold used inside a solver to stop an iteration. They also differ from an elapsed-time requirement. A solver may converge to a pose within its internal threshold yet return too late, or it may stop without convergence while retaining a configuration that meets the task. We distinguish solver-internal success, task-command admissibility, and within-deadline admissible success. This distinction makes the application requirement explicit without prescribing one numerical algorithm. It also permits a fair comparison between a solver using componentwise error bounds and one using error norms, provided that the relationship between those native conditions and the public task contract is documented and every returned command is independently checked. A bound on an individual iteration’s step is also not a bound on the displacement accumulated over an entire solve.

A mismatch between these layers has two practical consequences. An unnecessarily strict internal pose condition can spend iterations refining a command that is already admissible, or exhaust a search budget without returning an available task-level solution. An internal success can also fail the application’s joint or rate constraints if those constraints were absent or represented differently inside the search. Treating all negative return codes as infeasibility hides the first problem; treating all positive return codes as executable commands hides the second. Solver convergence is not command admissibility. For a failed query, a saved legal configuration under the identical target, previous state and command interval can establish that an admissible command was missed. Without such a witness, failure remains a statement about the particular numerical search. This is especially important in a trajectory: a feasible reference path proves that one sequence exists, but a solver’s own accepted states can depart from that path. A later failure from a different state does not by itself reveal whether the cause is insufficient search, an unfavorable configuration, or a genuinely empty next-command set.

We study these issues through a task-contract-aware solver interface. The task first defines a public acceptance set. The interface then maps its pose allowances conservatively into solver-native conditions and intersects the robot’s joint ranges with the displacement interval allowed from the actual previous command. A numerical solver searches this domain, after which an independent verifier evaluates its returned configuration against the original contract. The verifier, rather than the native return code, governs acceptance. We instantiate this sequence with the official Cartesian-bound capability of TRAC-IK and the norm-based stopping conditions of an existing Adaptive DLS implementation. The two instances deliberately retain their established search mechanisms. Their native acceptance sets are not assumed to be identical: TRAC-IK’s component box is an inscribed approximation to the public norm balls, whereas DLS tests the norms directly. The contribution is a common formulation, an explicit mapping and outcome interface, and evidence about its computational and closed-loop consequences. It does not depend on introducing another IK solver or learning a new validity rule.

This interface also changes how adaptive computation should be evaluated. Per-instance algorithm selection is useful when alternative computational paths have predictably different costs [12, 13]. For IK, choosing an initial seed or a refinement entry can improve some queries [14]; learned proposals can also supply configurations for numerical refinement. However, selection itself has a cost. Its apparent benefit can depend on a strict baseline spending time on unnecessary precision, on different implementations accepting different commands, or on avoiding work for requests that no compared policy completes. We therefore treat learned computation allocation as a secondary layer and examine a frozen CG-HIK portfolio as an empirical case study. Its purpose is to test the value left for entry selection and early rejection once convergence, admissibility and timing are distinguished. This ordering preserves the useful role of learned or hybrid IK without assuming that prediction is needed for every query. It also separates savings on successful command generation from savings on unsuccessful numerical work, which can imply different engineering choices even when their aggregate latency reductions look similar.

The paper makes three contributions. First, it formulates online IK under a joint pose-and-rate command contract and defines an outcome taxonomy that distinguishes native convergence, admissible returns and deadline-qualified returns, including exact-input witness-confirmed misses. Second, it provides a task-contract-aware interface for two existing solver families, documenting conservative native tolerance mapping, dynamic joint domains and independent acceptance. Third, it presents a two-robot empirical study covering witnessed point queries, separate position and orientation alignment, search budgets, contract-scale sensitivity and complete online trajectories, followed by an allocation-boundary analysis using frozen results. The point study separates current-query semantics from closed-loop effects; the trajectory study tests whether locally admissible numerical choices sustain tracking. Both improvements and losses are retained, so the evidence can identify where alignment is useful and where early stopping remains insufficient. The resulting engineering principle is to align the solver with the task contract before introducing adaptive computation, and then judge any additional decision layer by the remaining workload rather than by an assumed need for greater algorithmic complexity.

2 Related Work

2.1 Numerical and constrained inverse kinematics

Classical DLS methods address singularity-sensitive Jacobian inversion by trading exact residual reduction against bounded numerical steps [5–7]. TRAC-IK combines a modified Newton-style search with nonlinear optimization [8]. Current numerical work includes variable step-size Jacobian iteration [15], structure-exploiting differential IK [9], and modular kinematic optimization [16]. Analytical alternatives exploit geometric sub-problems or automated decomposition [3, 4]. GeoFIK studies Franka-specific redundancy parameterizations [17], while the HJCD-IK preprint combines batched coordinate-descent initialization with Jacobian polishing [18]. IKSPARK investigates convex-relaxation formulations with obstacle constraints [19]. These approaches improve numerical search or the representation of constraints. Our comparison instead keeps two numerical implementations fixed and varies their interface to a stated task-level pose contract.

2.2 Set-based and tolerance-aware robot tasks

Task Space Regions represent sets of allowed end-effector poses for manipulation planning [11]. Set-based task-priority IK explicitly accommodates variables that need to stay within an interval rather than attain a single value [10]. RelaxedIK optimizes accurate and feasible arm motion through a combination of task and motion objectives [2], and smooth task switching addresses continuity when task priorities change [20]. These works establish that tolerated deviation and constraint sets are longstanding robotics concepts. The present interface uses them at the boundary between an application and a solver: a public norm-based pose allowance must be translated into the solver’s own residual coordinates, bounds and stopping tests. A soft optimization objective, a componentwise acceptance box and a norm-ball verifier have different meanings even when all use a quantity called a tolerance.

2.3 Online IK, continuity, and joint-rate constraints

Redundant kinematic control has long used secondary objectives to manage configuration evolution [1]. Predictive online IK anticipates upcoming motion demands [21]. Trajectory scaling and predictive quadratic optimization offer ways to handle motion constraints [22, 23]; look-ahead optimization manages nullspace motion in dual-arm impedance control [24]. Viability-constrained IK explicitly seeks to retain feasible motion under joint range, velocity and acceleration constraints [25]. Singularity transitions in UR-like mechanisms [26] concern a distinct geometric structure from the redundant configuration evolution of a Panda arm. CppFlow searches and refines candidate joint trajectories for Cartesian paths [27]. Our online measurements neither optimize a future window nor rescale the target sequence. They test current-target solver interfaces while keeping the actual accepted-state feedback, target timing and final verifier common. This exposes continuity effects without attributing every failure to a branch transition or proposing a preview controller.

2.4 Adaptive computation and solver selection

Algorithm selection chooses a computational procedure conditional on the instance [12, 13]. IKSel applies informed seed selection and reselection to numerical IK [14], while ETA-IK considers motion execution time in dual-arm redundancy resolution [28]; motion execution time is different from solver latency. Learning-based IK can generate diverse or structured proposals, as in IKFlow [29], CycleIK [30], generative graphical IK [31], and learning-by-example methods [32]. Recent set prediction and the MimicIK preprint address multiple solutions and teleoperation-conditioned generation, respectively [33, 34]. Hybrid numerical refinement of learned approximations is also studied in an industrial-arm setting [35]. These developments motivate measuring proposal and selection costs as part of the full interface, while leaving acceptance to numerical verification.

Existing research thus separately covers task tolerance, constrained IK, continuous motion and algorithm selection. This study examines their interface: how solver-native convergence relates to the task-level command contract, and how that relationship affects missed admissible returns, computation and the interpretation of adaptive allocation. The empirical scope is two established solvers, not a claim to be the first use of task tolerances or solver bounds.

3 Problem Formulation

Let $f(q) = (p(q), R(q))$ denote forward kinematics for a bounded-joint manipulator, with $R(q) \in SO(3)$. An online query is

$$\zeta_t = (x_t^d, q_{t-1}, \Delta t), \quad x_t^d = (p_t^d, R_t^d), \quad (1)$$

where q_{t-1} is the actual last accepted configuration, not a reference state substituted after failure. The public command contract is

$$\begin{aligned} V(q; \xi_t) = 1 \iff & \|p(q) - p_t^d\|_2 \leq \epsilon_p, \quad d_R(R(q), R_t^d) \leq \epsilon_R, \\ & q_{\min} \leq q \leq q_{\max}, \\ & |q_i - q_{t-1,i}| \leq \dot{q}_{\max,i} \Delta t + \epsilon_v \quad \text{for every joint } i, \\ & \text{all evaluated values are finite,} \end{aligned} \quad (2)$$

with $d_R(R, R_d) = \|\text{Log}(R_d^\top R)^\vee\|_2$. Angles are in radians and positions in metres. The nominal contract uses $\epsilon_p = 1$ mm, $\epsilon_R = 0.5^\circ$, $\Delta t = 20$ ms and $\epsilon_v = 0.0001$ rad, together with the original per-joint URDF limits. Equation (2) gives the mathematical joint interval; the unchanged numerical verifier retains its pre-existing 10^{-9} rad joint-bound comparison allowance. Dynamic search domains use the strict URDF intersection. This distinction is recorded rather than silently changing the numerical acceptance boundary.

For solver s , write a call as

$$(\hat{q}_t, c_t, \tau_t) = s(\xi_t), \quad I_t = \mathbb{1}\{c_t \text{ is native success}\}, \quad A_t = V(\hat{q}_t; \xi_t), \quad D_t = A_t \mathbb{1}\{\tau_t \leq T\}. \quad (3)$$

Here I_t denotes solver-internal success, A_t denotes task admissibility, and D_t denotes within-deadline admissible success for $T = 20$ ms. A missing or non-finite return has $A_t = 0$. The time τ_t covers the full query interface, including conversion, dynamic bounds and final verification. An admissible return with $I_t = 0$ is usable under this contract; $I_t = 1$ alone never authorizes a command.

A *witness-confirmed missed-admissible query* has $A_t = 0$ for the measured call and a separately retained q_t^w satisfying $V(q_t^w; \xi_t) = 1$. The witness must use the identical target, previous state, interval and verifier. It establishes existence for this input, not a global property of the solver. Conversely, an unsuccessful local or timed search without such evidence does not establish mathematical infeasibility.

For a trajectory, geometric completion requires $\prod_t A_t = 1$ and deadline completion requires $\prod_t D_t = 1$. A reference-feasible path consists of a saved sequence satisfying the contract from its own preceding states. It does not establish feasibility from a different solver-generated state. In our geometric feedback loop an accepted return updates q_{t-1} ; a rejected return leaves it unchanged while the external target index advances. Geometrically admissible but late returns update this same loop and are counted as late. Deadline-triggered actuator behavior is outside this evaluation.

4 Task-Contract-Aware Solver Interface

4.1 Task-level command contract

An interface needs to know what constitutes a useful return before it can decide how much numerical accuracy to request. We express that requirement as Equation (2), independently of the solver and any learning component. The contract is an acceptance set rather than a cost function. This allows a return with a small but nonzero residual to be judged on its task meaning, while keeping joint motion and timing visible as separate conditions. Pose allowances are supplied by the application; no experimental outcome is used to choose them.

4.2 Solver-native contract mapping

Solvers represent residuals differently, so passing the same scalar tolerance to different APIs does not necessarily define the same problem. TRAC-IK uses componentwise `KDL::Twist` bounds. In the verified implementation, the relative error is expressed in the target frame:

$$e_p^K = (R_t^d)^\top (p(q) - p_t^d), \quad e_R^K = \text{Log}((R_t^d)^\top R(q))^\vee. \quad (4)$$

The rotational quantity is an axis-angle vector, not roll, pitch and yaw. Orthogonal rotation preserves the translation norm, and the rotation-vector norm agrees with the public geodesic distance.

For fully aligned TRAC-IK, each translation and rotation component receives

$$b_p = \text{nextafter}(\epsilon_p / \sqrt{3}, 0), \quad b_R = \text{nextafter}(\epsilon_R / \sqrt{3}, 0). \quad (5)$$

Thus, a vector lying inside the corresponding component box also lies inside the task norm ball. This is a conservative inner approximation: some task-admissible residual vectors lie outside the box. Zero additional bounds define the strict setting. Position-only and orientation-only settings apply Equation (5) to just that part.

The original internal epsilon remains 10^{-5} . The native bounded-error test zeros components inside their specified bounds before comparing the retained error with epsilon. Its effective per-component threshold is

Table 1: Nominal task-to-native pose mapping, identical for both robots. TRAC-IK columns give each component’s additional bound; zero retains its 10^{-5} internal epsilon. DLS columns give separate norm stopping thresholds. The public contract remains 1 mm and 0.5° for every row.

Setting	Position (mm)	Orientation (rad)	Native set	Budget
TRAC strict 5 ms	0.000000	0.00000000	Component box	5 ms
TRAC position 5 ms	0.577350	0.00000000	Component box	5 ms
TRAC orientation 5 ms	0.000000	0.00503833	Component box	5 ms
TRAC aligned 5 ms	0.577350	0.00503833	Component box	5 ms
TRAC strict 20 ms	0.000000	0.00000000	Component box	20 ms
TRAC aligned 20 ms	0.577350	0.00503833	Component box	20 ms
DLS strict	0.010000	0.00001000	Separate norms	25 iter.
DLS aligned	1.000000	0.00872665	Separate norms	25 iter.

$\max(b, \epsilon_{\text{int}})$, not $b + \epsilon_{\text{int}}$. The norms of the effective component-limit vectors are checked against the public tolerances at every prescribed scale. All aligned bounds exceed the internal epsilon here. Independent final verification handles numerical boundary effects and remains necessary even with a conservative mapping.

DLS computes separate position and orientation norms directly. Its aligned stopping conditions are the two public pose tests; its strict conditions are 10^{-5} m and 10^{-5} rad. The damping, singular-value threshold, orientation weight, line search, maximum update, minimum improvement and stagnation rule are unchanged. Table 1 gives the native settings. These are uses of existing Cartesian bounds and DLS stopping conditions, not new numerical algorithms.

4.3 Dynamic joint and joint-rate bounds

Joint-range constraints alone cannot enforce an online displacement allowance. For every query, the interface forms

$$\ell_i = \max(q_{\min,i}, q_{t-1,i} - a_i), \quad u_i = \min(q_{\max,i}, q_{t-1,i} + a_i), \quad a_i = \dot{q}_{\max,i} \Delta t + \epsilon_v. \quad (6)$$

TRAC-IK receives this dynamic joint interval before numerical search. A kinematics wrapper restricts DLS clipping to the same interval. Both start from the actual previous configuration. A representable interior construction handles floating-point endpoints consistently, moving a computed endpoint inward when necessary to satisfy the original displacement comparison. It does not increase the verifier tolerance. The final total displacement is still checked: clipping one numerical iteration is not a substitute for constraining the command’s displacement from q_{t-1} .

4.4 Independent command verification

A solver’s convergence report summarizes its own computation, whereas Equation (2) determines acceptance. The interface retains the raw configuration and native status, evaluates nonlinear forward kinematics, and checks every public condition independently. Only $A_t = 1$ authorizes a state update, including when the native status is failure. Rejection records preserve the residuals and specific finite-value, pose, joint or rate reason. A failed candidate is not projected, retried or replaced by a reference witness after the measured call. The recorded time extends through this final verification.

4.5 Outcome taxonomy and witness analysis

Figure 1 separates the four native/task combinations. The internal-failure/task-accept cell measures disagreement in return semantics. Witness-confirmed misses require an additional existence check and can occur with either native status when no command is accepted. For repeated searches, we distinguish UIDs with any failed call, UIDs with all calls failed and the mean missed-call count per sweep. These quantities answer different questions about input coverage and search variability. None treats repeated calls as independent new queries.

4.6 Adaptive computation as a secondary layer

After specifying acceptance, an optional policy can allocate numerical effort. The frozen CG-HIK case study selects among entries of a shared numerical portfolio using query-conditioned cost predictions. Action-complete observations measure all entries on a common input, avoiding a manually assigned difficulty label. Predicted tail cost includes fallback work, and a shared terminal-success target reflects the common acceptance semantics.

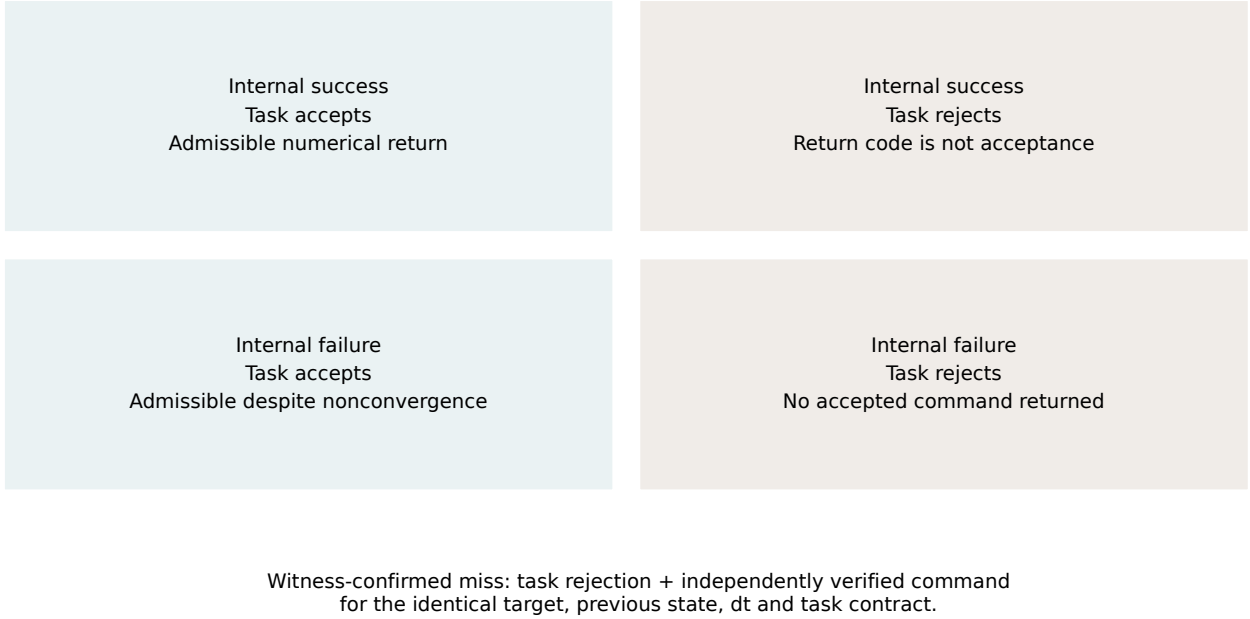


Figure 1: Solver-internal success and task admissibility form independent outcome axes. Deadline qualification is an additional test on accepted returns. A witness-confirmed miss requires a legal configuration for the exact input; a failed numerical search alone is insufficient. This schematic defines the taxonomy and does not imply that every empirical cell is nonempty.

Reject avoids predicted futile computation; defer retains a conservative numerical path. Neither changes the public verifier.

The empirical controls isolate direct hard entry, a geometric entry rule, reject-only followed by hard, routing without reject, median-cost selection and the full policy. This comparison tests whether entry ranking and failure avoidance justify their prediction overhead. It uses frozen historical records; the current study neither trains a policy nor adds one to the aligned native solvers.

5 Experimental Protocol

5.1 Robots, numerical implementations, and common conditions

We use the seven-joint Panda and six-joint UR5e with their original URDF geometry, joint order and per-joint limits. The public forward-kinematics and Jacobian backend is the repository’s URDFKinematics; the native solver uses the matching KDL chain. Smoke checks confirm their pose agreement and the rotation-vector mapping. TRAC-IK is official version 2.2.0 in Speed mode with its original internal epsilon and two native search workers. The measured process uses the same Python 3.12.13 and NumPy 2.4.4 environment as the authoritative Panda trajectories, on an Intel Core i7-13700K workstation. BLAS and OpenMP thread counts are fixed to one. The Python scientific stack supports analysis and verification [36]; PyTorch and scikit-learn are used by the frozen allocation case study [37, 38]. No learned model is trained or changed in this study.

Six TRAC-IK point configurations are fixed: strict, position-aligned, orientation-aligned and fully aligned at 5 ms; strict and fully aligned at 20 ms. Two DLS configurations use strict or task pose stops with the same 25-iteration cap. A fixed iteration cap is not an equal wall-time budget. Actual return-within-budget proportions are recorded instead of assuming that DLS is a 5 or 20 ms solver.

5.2 Witness-feasible and inexecutable point queries

For each robot we reuse the previously fixed set of 2500 witness-feasible points and 500 constructed inexecutable points. The feasible families contain local, near-singular, near-limit and hard-feasible queries, with their original proportions. Every witness is checked under the exact input and public contract before measurement. The inexecutable targets exceed a conservative global reach bound obtained from URDF link translations; they are not labeled by solver failure. Their rejection and computation are reported separately.

All settings receive identical targets, previous configurations and intervals. Point calls do not update the previous state. TRAC-IK has three interleaved search repeats for each input and configuration; deterministic

DLS runs once. A predetermined shuffled order interleaves the eligible settings. The library’s random search is retained, without claiming control of its internal random seed. Raw records include native status, returned configuration, all public checks, residuals, deadline outcome and timing components.

For both DLS point settings, an observation wrapper records actual FK evaluations without changing numerical operations. It identifies main iterates and line-search evaluations separately. Offline verifier replay locates the first admissible main iterate, then counts the remaining iterations and evaluations before the strict solver stops. Both point settings pay the same logging instrumentation. Offline trace processing is excluded from solve latency; no unmeasured phase-specific time saving is inferred.

5.3 Contract-scale sensitivity

Before any sensitivity solve, we select 500 fixed witnessed queries per robot whose witnesses also pass the half-scale pose contract. Selection uses a fixed family allocation and UID ordering, not solver outcomes. The same inputs are evaluated at half, nominal and double position and orientation tolerances, keeping joint limits, velocity limits, velocity allowance, target poses, previous configurations and Δt unchanged. Strict TRAC-IK 5 ms, fully aligned TRAC-IK 5 ms and task-aligned DLS are the only settings. Native bounds follow Equation (5) at every scale. The nominal contract remains primary; a success gained by changing the public contract is not labeled a same-contract algorithmic improvement.

5.4 Complete continuous trajectories

Panda’s existing 40 reference-feasible trajectories and all of their measured runs are used unchanged. The symmetric UR5e experiment uses 40 new trajectories, ten each for smooth motion, near-singular motion, joint-limit return and high curvature. Each contains 150 frames at $\Delta t = 20$ ms. The same geometry-only generator first constructs a continuous reference joint path and then obtains targets through FK. Every reference transition passes the public contract; identities are fixed before solver outcomes. All trajectories are retained without success-based screening.

Every setting starts from the same initial reference configuration. For each target it receives only that target and its own previous accepted state. Future reference joints are stripped from the online input. Accepted commands update the state; rejected commands hold it. All methods still process all target frames, without restarting on the reference branch. Four TRAC-IK configurations (strict/aligned at both budgets) each have three full repeats; each DLS setting has one. The new UR5e runs reuse the identical wrapper and timing scope of the prior Panda study. No point-trace logging is enabled in either robot’s trajectory measurements.

5.5 Measurement, statistical units, and evidence use

Outer latency includes required input conversion, dynamic-bound construction and setup, numerical solving, final task verification and intervening interface costs. Initial library/chain loading and offline serialization are excluded. Point records separately retain conversion, bounds/setup, solve, verification and accounting remainder. Latency quantiles include failed returns; cumulative trajectory time includes every fixed target frame. Accepted position and orientation distributions are reported alongside success and cost. No failure, long call or unfavorable family is deleted or winsorized.

The independent unit is the query UID for points and the complete trajectory UID for tracking. Search repeats are averaged within each unit before paired success-difference and mean/cumulative-cost comparisons. We use 4,000 paired, family-stratified percentile bootstrap resamples for 95% intervals. Frame quantiles and within-trajectory search-repeat variation are descriptive; frames and repeated calls are not independent inferential samples. Zero-width empirical success intervals at a ceiling do not bound failure probability on other workloads. Family effects are reported in full without multiple-comparison significance claims.

The point inputs have been observed in earlier mechanism work, and the Panda trajectory outcomes predate the new supplementary experiment. The new UR5e input identities, mappings, budgets and analysis rules were fixed before its outcomes. This is a planned supplementary study with an authoritative historical arm, not a claim that all evidence is a fresh formal test. The historical allocation analysis retains its own frozen measurements and failure semantics. All new measurements were completed and published before manuscript reconstruction; no result triggered tuning or an additional solver.

Table 2: Witness-feasible point queries, 2500 independent queries per robot. Internal and verified columns are percentages. Miss is the number of UIDs with at least one missed-admissible call. TRAC-IK has three within-query repeats; DLS has one. All latency columns are milliseconds and include the final verifier. DLS point timings include the same trace instrumentation in both settings.

Robot	Setting	Internal	Verified	Miss	Mean	P50	P95	P99
Panda	TRAC strict 5 ms	100.00	100.00	0	0.236	0.211	0.367	0.487
Panda	TRAC position 5 ms	100.00	100.00	0	0.241	0.211	0.379	0.681
Panda	TRAC orientation 5 ms	100.00	100.00	0	0.247	0.210	0.382	0.799
Panda	TRAC aligned 5 ms	100.00	100.00	0	0.221	0.199	0.332	0.478
Panda	TRAC strict 20 ms	100.00	100.00	0	0.236	0.211	0.362	0.485
Panda	TRAC aligned 20 ms	100.00	100.00	0	0.220	0.199	0.331	0.467
Panda	DLS strict	77.56	99.48	13	3.015	1.038	8.944	9.230
Panda	DLS aligned	99.48	99.48	13	0.762	0.625	1.042	4.479
UR5e	TRAC strict 5 ms	100.00	100.00	0	0.218	0.201	0.318	0.409
UR5e	TRAC position 5 ms	100.00	100.00	0	0.216	0.199	0.317	0.414
UR5e	TRAC orientation 5 ms	99.99	99.99	1	0.219	0.200	0.324	0.445
UR5e	TRAC aligned 5 ms	100.00	100.00	0	0.209	0.191	0.312	0.415
UR5e	TRAC strict 20 ms	100.00	100.00	0	0.222	0.200	0.320	0.424
UR5e	TRAC aligned 20 ms	100.00	100.00	0	0.210	0.192	0.313	0.410
UR5e	DLS strict	78.72	99.96	1	2.757	1.185	8.113	8.427
UR5e	DLS aligned	99.96	99.96	1	0.656	0.578	0.926	1.458

6 Results

6.1 RQ1: How often do solver convergence and task admissibility disagree?

The first question is whether a native return flag correctly identifies commands that satisfy the public contract. The witnessed point population separates this question from uncertainty about whether an acceptable configuration exists. The complete status matrix is reported in Appendix A; Table 2 summarizes its task-level consequences. Each TRAC-IK setting has three calls per query, whereas each DLS setting has one.

The largest observed disagreement is an internal failure that still returns an admissible command. Strict DLS exhibits this outcome on 548 Panda queries (21.92%) and 531 UR5e queries (21.24%). Its internal success rates are therefore substantially below its task-admissible rates. Accepting only a positive convergence flag would discard these valid numerical returns. Task-aligned DLS removes this semantic disagreement on the measured points, but the actual missed-admissible counts remain 13 and 1. Changing the flag’s interpretation and recovering a previously unavailable command are different effects.

In contrast, strict and fully aligned TRAC-IK accept every witnessed nominal point in every repeat. Their internal and task outcomes agree on this population. Position-only alignment also retains all successes; orientation-only alignment has 1 failed call on 1 UR5e query with a valid witness. Thus, the component arms recover no strict failures here. The point data do not identify a position-driven or orientation-driven share of the later trajectory gains. There are no internal-success/task-reject calls in the new nominal point arms. The taxonomy accommodates that outcome without requiring every cell to be occupied.

The DLS traces explain a separate computational mismatch. Of the 2487 Panda and 2499 UR5e strict-DLS queries that reach an admissible main iterate, 2397 and 2346 continue iterating. The median is 1 excess iteration and the P95 is 24 for both robots (Appendix B). These are directly observed excess iterations after task admissibility, not an inference from separate runtimes. TRAC-IK does not expose a comparable trace.

All constructed inexecutable points are rejected by every setting, with no false acceptance. Independently replaying the point and trajectory returns finds 0 accepted contract violations. The observed disagreement is therefore concentrated in strict DLS’s interpretation of otherwise admissible returns; the next question is whether native tolerance alignment reduces computation under unchanged public acceptance.

6.2 RQ2: Does contract alignment improve task-level success and computation?

With acceptance held fixed, does using the available pose tolerance reduce work, and does any gain persist across contracts and solver budgets? Figure 2 shows all prespecified point settings. On the nominal witnessed points, fully aligned TRAC-IK at 5 ms reduces mean whole-call time by 6.41% on Panda and 4.00% on UR5e, while preserving complete success. The paired mean-cost ratios are 0.936 [0.928, 0.945] and 0.960 [0.949, 0.970], respectively (brackets denote 95% intervals). These are modest interface-level savings on a population that strict TRAC-IK already solves reliably.

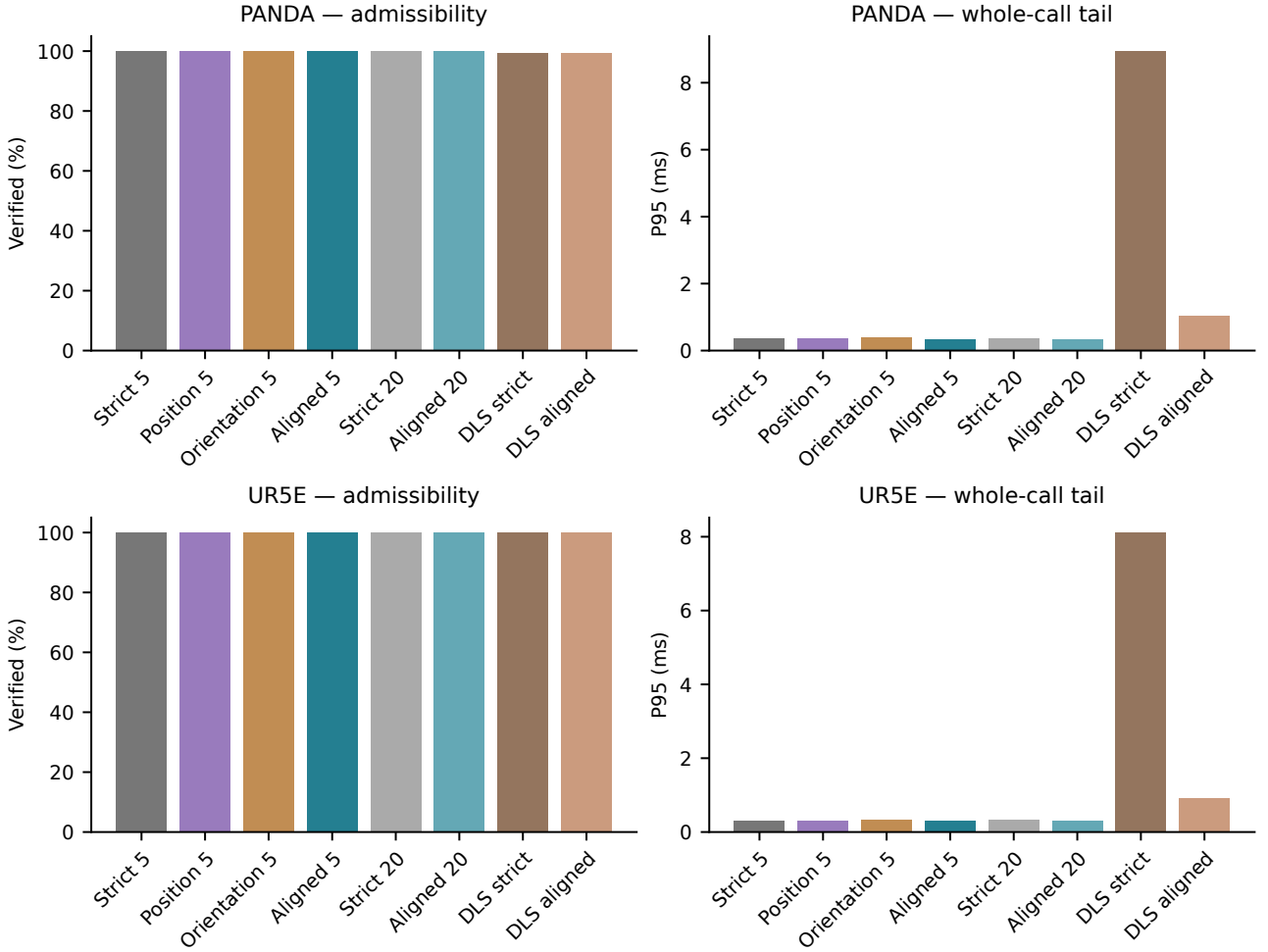


Figure 2: Nominal witnessed points for both robots. Verified success and whole-call P95 are shown separately for every prescribed setting. The near-ceiling success bars should be read with the exact counts in Table 2. The numbers 5 and 20 label TRAC-IK search budgets in milliseconds; DLS uses a fixed iteration cap. Bars are descriptive call summaries, not independent-call inference.

Partial alignment has no uniform cost advantage. On Panda, position-only and orientation-only cost ratios are 1.022 [1.011, 1.032] and 1.043 [1.025, 1.064]. The corresponding UR5e ratios are 0.989 [0.979, 0.998] and 1.006 [0.993, 1.019]. Tolerance dimensions interact with numerical search, so the effects of the component arms need not add to the fully aligned effect. Increasing strict TRAC-IK’s budget to 20 ms adds no point successes. Constructed inexecutable requests still consume approximately the search budget under either mapping; their cost is reported separately in the source tables.

Task-aligned DLS preserves the same point-level acceptance sets while reducing mean time by 74.72% and 76.19%. P95 falls from 8.944 to 1.042 ms on Panda and from 8.113 to 0.926 ms on UR5e. The direct trace evidence supports attributing this reduction to earlier stopping, within the instrumentation scope of these measurements. Absolute DLS and native TRAC-IK times also reflect different implementations and are not equal-time-budget comparisons.

The saved work accompanies larger legal errors. For fully aligned TRAC-IK at 5 ms, accepted position P95 rises from 0.0073 to 0.5760 mm on Panda and from 0.0075 to 0.5029 mm on UR5e. Accepted orientation P95 becomes 0.1397° and 0.0694°. The full error distributions, including maxima and DLS returns, accompany the source tables. These errors spend an existing task allowance; the final nominal limits have not changed.

Across the fixed contract scales, strict and fully aligned TRAC-IK remain fully successful (Figure 3). Panda task-aligned DLS misses 2 queries at the half-scale contract and none at nominal or double scale; UR5e misses none. Its mean times decrease as the permitted pose sets expand. Fully aligned TRAC-IK also generally obtains a larger relative mean-cost advantage at larger scales, although individual quantiles are not strictly monotonic. The scale study therefore supports a computational interpretation, not a broad claim of tolerance-dependent recovery for TRAC-IK on these points. Alignment reduces work under a fixed nominal contract in both implementations, but whether earlier accepted commands sustain a continuous trajectory requires a closed-loop test.

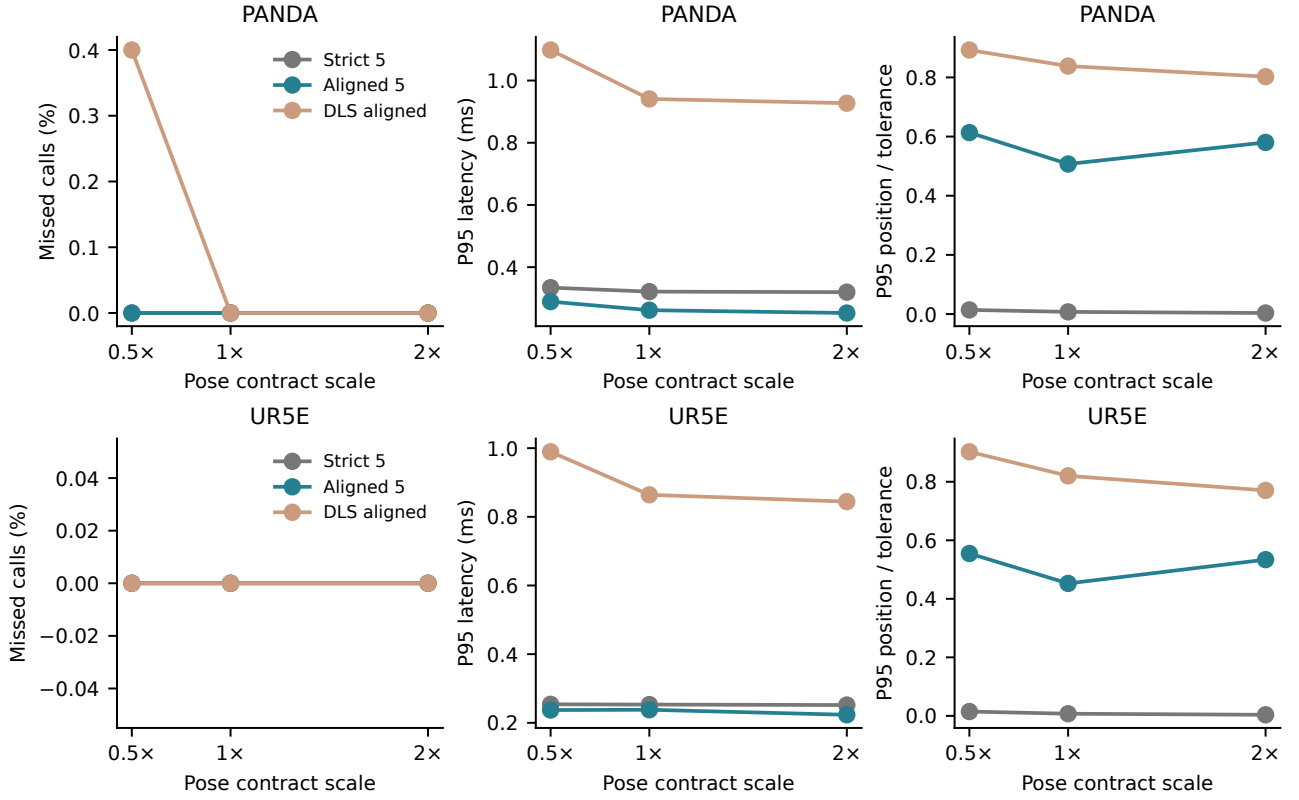


Figure 3: Contract sensitivity on the same 500 witnessed queries per robot at every scale. Only the two pose tolerances change. Missed-call rates, whole-call P95, and accepted position P95 divided by the applicable tolerance are shown. Overlapping zero-miss curves indicate complete observed success; they do not establish zero population failure probability. Orientation distributions are included in the accompanying source data.

6.3 RQ3: Does the effect persist in trajectories, and when is adaptive computation useful?

A faster admissible point return is useful online only if the resulting sequence continues to meet the contract. We therefore compare complete trajectories with each setting feeding back its own last accepted command. Table 3 and Figure 4 retain every target frame and every planned search repeat.

At 5 ms, Panda TRAC-IK completion increases from 34/34/34 to 36/37/37 out of 40 per sweep. UR5e increases from 37/37/37 to 39/39/39. No strict-completed UID is lost. The paired completion-rate differences are 6.667 [1.667, 13.333] percentage points for Panda and 5.000 [0.000, 12.500] for UR5e. Cumulative solver time decreases by 32.53% and 42.59%; paired cost ratios are 0.675 [0.482, 0.890] and 0.574 [0.330, 0.979]. The intervals reflect independent trajectory variation, not thousands of independent frames.

Larger search budgets do not substitute for the mapping. Strict TRAC-IK at 20 ms completes the same number of trajectories as strict 5 ms on both robots, while spending more time on unsuccessful requests. Aligned 20 ms sometimes completes more Panda paths than aligned 5 ms, but raises cumulative time; UR5e completion is unchanged. Whole-trajectory deadline success is separately listed in Table 3. Occasional late frames make it differ from geometric completion even when the nominal sampling interval is 20 ms.

The family analysis localizes the gains rather than hiding them in a total. Panda’s 5 ms recovery is confined to near-singular paths. UR5e gains 1 smooth and 1 high-curvature path, while its near-singular completion remains unchanged. Joint-limit-return UR5e time changes little, and Panda high-curvature time does not improve. Figure 5 and Appendix C include every family.

DLS exposes an important boundary. On Panda, both stopping rules complete 37 paths, and task alignment reduces cumulative time by 66.23%. On UR5e, completion falls from 35 to 28, losing 7 strict-completed paths despite saving 61.18% of cumulative computation. The completion difference is -17.500 [-30.000, -7.500] percentage points. The losses span smooth, near-singular and joint-limit-return families. Both settings still submit only admissible commands. Earlier stopping therefore changes more than time spent on the current query: it changes the configuration presented to subsequent queries. These trajectories do not isolate the cause of each failure or establish infeasibility from the failed setting’s state.

The residual role of computation allocation. The frozen CG-HIK case study asks whether additional entry selection compensates for its own overhead. Table 4 uses historical paired point measurements, not newly

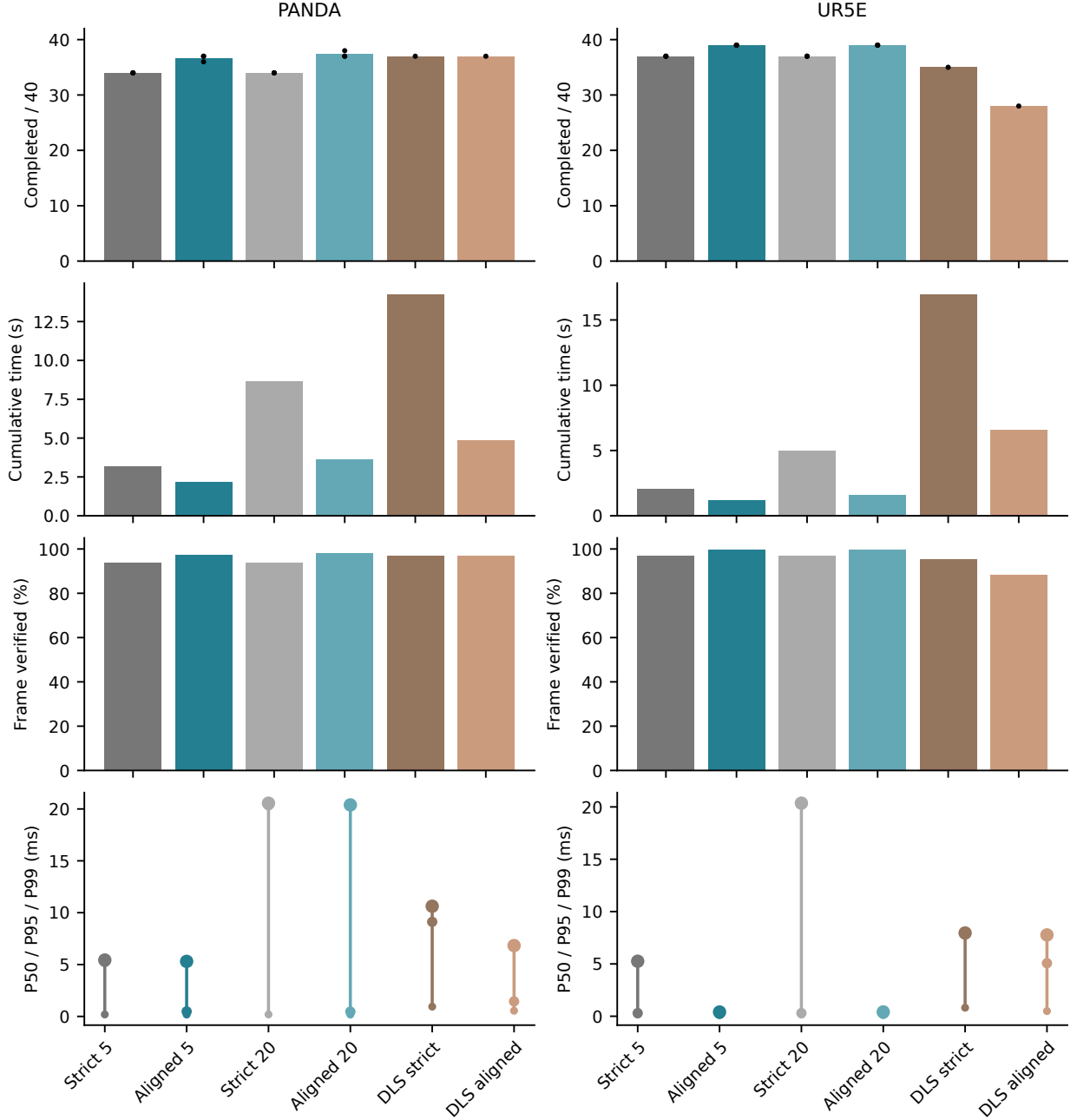


Figure 4: Continuous online operation. Columns separate robots; rows report geometric completion, cumulative cost, frame admissibility and latency quantiles. Dots on completion bars show the planned search repeats. In the bottom row, increasing marker sizes denote P50, P95 and P99; connecting segments only group the three quantiles of one setting. All fixed target frames contribute to cost. UR5e’s lower DLS-aligned completion is retained alongside its lower cost.

Table 3: Complete online trajectories. Each robot has 40 independent paths with 150 targets each. Complete and deadline columns give counts per planned sweep; the latter requires every frame to be admissible and returned within 20 ms. Frame is the verified percentage. Time is cumulative seconds per full sweep, averaged over the three TRAC-IK sweeps. Latency quantiles are in milliseconds. Panda measurements are the unchanged authoritative prior runs.

Robot	Setting	Complete	Deadline	Frame	Time	P50	P95	P99
Panda	TRAC strict 5 ms	34/34/34	34/34/33	93.98	3.187	0.190	5.249	5.434
Panda	TRAC aligned 5 ms	36/37/37	36/36/37	97.28	2.150	0.179	0.484	5.305
Panda	TRAC strict 20 ms	34/34/34	34/34/34	93.99	8.654	0.192	20.352	20.559
Panda	TRAC aligned 20 ms	37/37/38	37/37/38	98.16	3.600	0.180	0.462	20.392
Panda	DLS strict	37	37	96.80	14.255	0.943	9.119	10.624
Panda	DLS aligned	37	37	96.80	4.814	0.550	1.451	6.830
UR5e	TRAC strict 5 ms	37/37/37	37/37/37	96.82	2.032	0.169	0.300	5.256
UR5e	TRAC aligned 5 ms	39/39/39	39/39/39	99.57	1.167	0.161	0.223	0.405
UR5e	TRAC strict 20 ms	37/37/37	37/36/37	96.82	4.932	0.169	0.303	20.357
UR5e	TRAC aligned 20 ms	39/39/39	39/39/38	99.58	1.568	0.161	0.229	0.412
UR5e	DLS strict	35	35	95.32	16.965	0.816	7.813	7.955
UR5e	DLS aligned	28	28	88.37	6.586	0.495	5.066	7.769

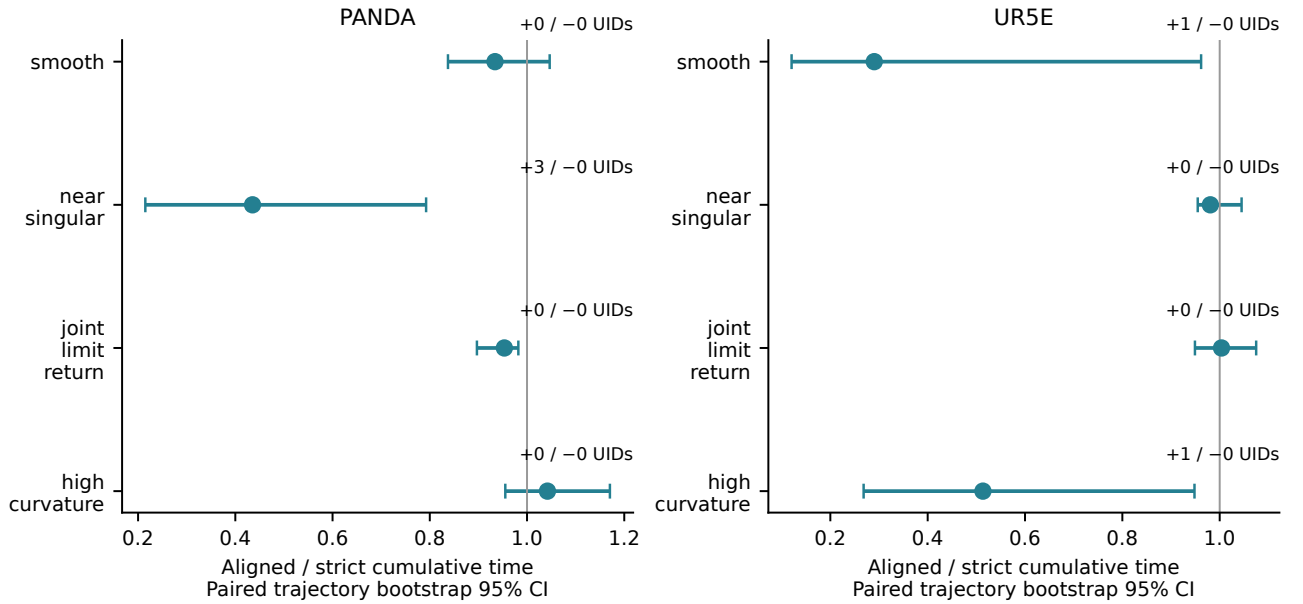


Figure 5: Fully aligned versus strict TRAC-IK at 5 ms, by trajectory family. Intervals resample paired trajectory UIDs within the stated family. Annotations count UIDs with increased/decreased completion fractions across repeats, not independent new paths; Panda’s gains include a partial search-repeat recovery. The reference line denotes unchanged cumulative cost.

routed calls to the aligned solvers. On witnessed points, full CG-HIK matches direct hard-entry success but increases mean time by 3.33% on Panda and 29.32% on UR5e. Routing-only is also slower, and a geometric entry rule is close to the full policy. Selecting by predicted P95 rather than P50 reduces function evaluations by 0.58% and 1.86%, but its P95 and P99 ratio intervals include unity on both robots.

Post-hoc decomposition of the original transition-rich workload assigns 99.07% and 101.31% of net time savings to paths neither method completed. A share above one hundred percent means that savings in that group offset additional cost elsewhere. Independently witnessed reference paths likewise provide no universal successful-tracking advantage for learned routing. Reject-only controls show that avoiding unsuccessful numerical work is useful on failure-heavy requests. The historical external TRAC-IK comparison is much faster than this learned/numerical portfolio; its documented floating-point endpoint issue remains an implementation caveat, not a newly attributed tolerance-alignment benefit.

Taken together, continuous trajectories support the aligned TRAC-IK interface on both tested robots while revealing a DLS stopping-rule trade-off that points alone miss. The allocation evidence favors paying for a secondary decision layer only when the remaining unsuccessful-solve cost is large enough to justify it.

Table 4: Secondary allocation case study on the same historical witnessed point population. Ratios compare the named policies; brackets are paired query-level 95% intervals. The success difference is in percentage points. Geometry and reject-only controls retain the same reject/defer predictor. These are not new measurements of a router added to aligned TRAC-IK or DLS.

Robot	Contrast	Mean time ratio	FEV ratio	Success diff.
panda	Full / hard	1.033 [1.029, 1.038]	1.001 [1.001, 1.002]	0.00
panda	Routing-only / hard	1.033 [1.030, 1.038]	1.001 [1.001, 1.002]	0.00
panda	Full / geometry	1.001 [1.000, 1.002]	1.001 [1.000, 1.001]	0.00
panda	Full / reject-only	1.000 [0.998, 1.001]	1.001 [1.001, 1.002]	0.00
panda	P95 / P50 selection	0.998 [0.997, 1.000]	0.994 [0.992, 0.996]	0.00
ur5e	Full / hard	1.293 [1.286, 1.300]	1.000 [1.000, 1.001]	0.00
ur5e	Routing-only / hard	1.294 [1.286, 1.300]	1.000 [1.000, 1.001]	0.00
ur5e	Full / geometry	1.002 [0.998, 1.005]	0.999 [0.996, 1.000]	0.00
ur5e	Full / reject-only	1.002 [0.998, 1.005]	1.000 [1.000, 1.001]	0.00
ur5e	P95 / P50 selection	0.998 [0.994, 1.002]	0.981 [0.974, 0.989]	0.00

7 Discussion

7.1 A return code answers a different question from a task verifier

A native convergence flag describes whether a numerical procedure met its configured conditions. The public verifier describes whether its returned configuration meets the command contract. Strict DLS demonstrates the practical difference: many negative flags accompany legal returns, and its observed iteration traces show substantial work after admissibility. TRAC-IK’s agreement on the witnessed point population is equally informative. An explicit taxonomy helps identify when disagreement matters; it does not require disagreement to be common in every solver.

Witnesses strengthen failure interpretation only when they refer to an identical input. The point dataset supplies that comparison directly. Reference paths in the trajectory study supply a weaker existence statement: they verify the reference sequence, not every state visited by another solver. Accordingly, improved closed-loop completion is evidence about the interface and its induced trajectory, while individual failures remain numerical outcomes unless an exact-input witness has been saved.

7.2 Alignment can remove work, but earlier acceptance changes feedback

The DLS trace supplies direct evidence for excess iterations after task admissibility. Allowing termination at the public pose threshold reduces that work on both point populations. Fully aligned TRAC-IK has smaller point-level cost benefits and larger continuous-trajectory benefits. A longer strict search budget does not recover the same completions, which argues against attributing the trajectory difference solely to too little wall time. Because its internal search path is not observed, the TRAC-IK measurements establish outcomes rather than a count of unnecessary internal iterations.

The UR5e DLS result limits the interpretation of earlier stopping. An admissible set can contain configurations with different subsequent tracking behavior. Stopping at its first reached point changes the feedback state and may preserve less useful pose accuracy or joint-space geometry for the next target. The present comparison changes those quantities together; it does not identify either as the sole cause. Thus, contract-aware acceptance is a sound interface requirement, but choosing the earliest admissible iterate is not guaranteed to preserve whole-trajectory completion. The difference between the two robots is part of the finding: contract mismatch and the benefit of its correction are robot-, solver- and workload-dependent.

7.3 Using a task allowance is not loosening the final task

The strict/aligned comparisons retain identical public position, orientation, joint and rate tests. Task-aligned returns have larger residuals because they use an allowance already present in that contract. Reporting those distributions makes the precision–computation trade-off visible without treating all accepted commands as equally accurate. For an application that requires tighter pose accuracy, the correct response is a tighter public contract and a corresponding native mapping, not an undocumented strict threshold inside one baseline.

The sensitivity study makes that distinction explicit. It changes the public pose contract in separate, prespecified arms while preserving the same witnessed inputs and motion limits. Increasing the permitted pose set generally reduces aligned computation, but the nominal point TRAC-IK success ceiling leaves no recovery trend to estimate. Moreover, the inscribed native box may exclude legal norm-ball residuals. Conservativeness

protects the interpretation of native bounds; it does not make the two admissible sets equivalent or ensure that the solver finds all legal commands.

7.4 Allocation should be judged after the numerical interface

The historical allocation study shows why aggregate speed alone can mislead solver-design decisions. Avoiding futile fallback work can save considerable time even when successful paths do not become faster. A learned entry policy can also reduce numerical evaluations yet lose that saving to inference and interface overhead. These are useful distinctions for engineering a service that processes both feasible and failure-heavy requests.

For the measured portfolio, complex routing does not generally accelerate successful IK, and simple entry controls are competitive. Reject remains useful when unsuccessful numerical work is expensive. The current native comparison does not directly measure a router added to an aligned solver, so its marginal value in that setting is an empirical boundary rather than a new measured speedup. The supported ordering is nevertheless concrete: first establish a common command contract, configure the numerical interface to use it, and then compare the remaining failed-request cost with the full overhead of any adaptive decision. Better configuration of a mature solver is an engineering result, not a reason to rename that solver as a new algorithm.

8 Limitations

The study uses exact software kinematics on two manipulators. It excludes collision checking, dynamics, contact, torque limits, calibration uncertainty and physical controller behavior. Geometric admissibility under the stated contract is not physical safety certification. The 20 ms deadline is an evaluation endpoint; nominal sampling at that interval does not establish hard real-time control.

TRAC-IK and the existing Adaptive DLS are the only newly measured solvers. Their native acceptance regions differ, with the component box conservatively approximating a norm ball. Search budgets and iteration caps are not equivalent resources. Absolute latency depends on hardware, native/Python implementation, threading, operating-system scheduling and the shared workstation. Both DLS point settings include trace logging, whereas trajectories use the same uninstrumented wrapper across robots.

The supplementary point workload is fixed and previously observed, and its TRAC-IK success ceiling limits recovery analysis. Panda’s authoritative trajectories were measured earlier than the symmetric UR5e runs. The paired intervals quantify variation within the sampled families, not coverage of all online IK tasks. A reference-feasible path does not guarantee a legal continuation from every method-specific state. Finally, the allocation boundary uses historical controls rather than a new factorial experiment adding a router to each aligned solver. The conclusions therefore distinguish directly measured alignment effects from the engineering judgment about when an additional decision layer is worth its cost.

9 Conclusions

Solver convergence is not command admissibility. A common task contract separates native convergence, legal command generation and deadline-qualified success, and provides a basis for configuring existing solver interfaces. In this study, task-aligned TRAC-IK preserves point success, improves complete tracking on both robots and reduces cumulative trajectory computation under the same final acceptance conditions. DLS exposes frequent internal-failure/admissible-return disagreement and excess iterations, but its UR5e completion loss shows that earlier local acceptance is not a universal closed-loop improvement. The frozen allocation case study further places most aggregate benefit in failure handling rather than broadly faster successful entry selection.

The resulting design principle is to align the solver with the task contract before introducing adaptive computation. Any subsequent allocation layer should earn its overhead from the residual workload, with success, precision and complete-trajectory behavior reported alongside time.

Data and code availability

The implementation, fixed protocol, input identities, raw returned configurations, timing records, aggregate tables and verified witness indices are available in the public repository <https://github.com/dingzhen60027/CG-HIK>, branch codex/hierarchical-v5. The paper’s evidence snapshot identifies the frozen measurement commit and file hashes. Manuscript numbers and figures are generated from those records without solving new queries. Historical evidence and the previous manuscript are retained separately. The repository supports inspection of the reported software benchmark; it is not a hardware deployment package.

A Complete nominal point outcome matrix

Table 5 gives raw call counts for the witnessed nominal population. The independent sample size remains 2500 query UUIDs per robot. Every internal-failure/task-reject entry in this population has a saved legal witness under the identical input. Counts across search repeats do not represent additional independent queries.

Table 5: Native/task outcome matrix for witnessed nominal points. I is native success and A is task acceptance. Three TRAC-IK calls and one DLS call per UUID are retained; counts are not pooled into a larger statistical sample. Constructed inexecutable points are reported separately in source data.

Robot	Setting	$I=1, A=1$	$I=1, A=0$	$I=0, A=1$	$I=0, A=0$
panda	TRAC position 5 ms	7500	0	0	0
panda	DLS strict	1939	0	548	13
panda	TRAC aligned 5 ms	7500	0	0	0
panda	TRAC strict 5 ms	7500	0	0	0
panda	TRAC aligned 20 ms	7500	0	0	0
panda	TRAC strict 20 ms	7500	0	0	0
panda	TRAC orientation 5 ms	7500	0	0	0
panda	DLS aligned	2487	0	0	13
ur5e	TRAC position 5 ms	7500	0	0	0
ur5e	DLS strict	1968	0	531	1
ur5e	TRAC aligned 5 ms	7500	0	0	0
ur5e	TRAC strict 5 ms	7500	0	0	0
ur5e	TRAC aligned 20 ms	7500	0	0	0
ur5e	TRAC strict 20 ms	7500	0	0	0
ur5e	TRAC orientation 5 ms	7499	0	0	1
ur5e	DLS aligned	2499	0	0	1

B Directly observed DLS excess iterations

The observation wrapper records the unchanged solver’s FK calls and identifies main iterates by their call site. A complete residual/evaluation trace remains available for every strict point query, including those that never produce an admissible iterate. Figure 6 conditions only the excess-iteration quantity on its being defined. The full population is retained in the source table; a query without an admissible iterate is not assigned zero excess work. No comparable unobserved TRAC-IK iteration quantity is inferred.

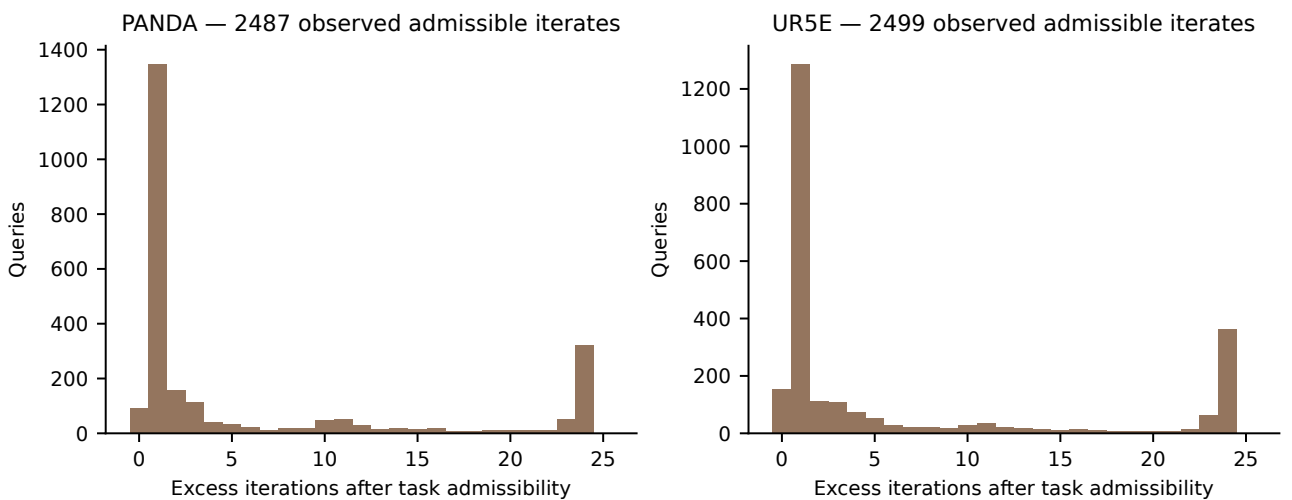


Figure 6: Excess iterations after task admissibility in strict DLS. Each included query contributes its stopping iteration minus its first admissible main-iterate index. Panel titles count queries with such an iterate, not independent iteration samples. Remaining queries are retained with undefined excess values in source data. Phase-specific extra time is not estimated.

C Complete family-level trajectory effects

Table 6: Fully aligned/strict TRAC-IK at 5 ms for all trajectory families. Each family contains ten independent paths. Completion lists correspond to three complete search repeats; intervals use paired trajectory resampling. Gained/lost counts compare per-UID completion fractions.

Robot	Family	Strict	Aligned	Cost ratio [95% CI]	Gain/loss
panda	high curvature	10/10/10	10/10/10	1.042 [0.955, 1.170]	0/0
panda	joint limit return	9/9/9	9/9/9	0.953 [0.897, 0.982]	0/0
panda	near singular	5/5/5	7/8/8	0.435 [0.215, 0.792]	3/0
panda	smooth	10/10/10	10/10/10	0.934 [0.837, 1.047]	0/0
ur5e	high curvature	9/9/9	10/10/10	0.514 [0.268, 0.948]	1/0
ur5e	joint limit return	10/10/10	10/10/10	1.004 [0.949, 1.075]	0/0
ur5e	near singular	9/9/9	9/9/9	0.981 [0.955, 1.045]	0/0
ur5e	smooth	9/9/9	10/10/10	0.290 [0.120, 0.962]	1/0

The distributed CSV/JSON tables additionally provide per-trajectory cumulative mean, median and P95, completion UID sets, accepted-error distributions, first-failure inputs and reasons, and repeated-search variability. All full trajectories and returned configurations can be inspected through the witness and run indices. No conclusion requires selecting only the successful trajectories.

References

- [1] Bruno Siciliano. Kinematic control of redundant robot manipulators: A tutorial. *Journal of Intelligent and Robotic Systems*, 3(3):201–212, 1990. doi: 10.1007/BF00126069.
- [2] Daniel Rakita, Bilge Mutlu, and Michael Gleicher. RelaxedIK: Real-time synthesis of accurate and feasible robot arm motion. In *Robotics: Science and Systems XIV*, 2018. doi: 10.15607/RSS.2018.XIV.043.
- [3] Alexander J. Elias and John T. Wen. IK-Geo: Unified robot inverse kinematics using subproblem decomposition. *Mechanism and Machine Theory*, 209:105971, 2025. doi: 10.1016/j.mechmachtheory.2025.105971.
- [4] Daniel Ostermeier, Jonathan K lzl, and Matthias Althoff. Automatic geometric decomposition for analytical inverse kinematics. *IEEE Robotics and Automation Letters*, 10(10):9964–9971, 2025. doi: 10.1109/LRA.2025.3597897.
- [5] Charles W. Wampler, II. Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods. *IEEE Transactions on Systems, Man, and Cybernetics*, 16(1):93–101, 1986. doi: 10.1109/TSMC.1986.289285.
- [6] Yoshihiko Nakamura and Hideo Hanafusa. Inverse kinematic solutions with singularity robustness for robot manipulator control. *Journal of Dynamic Systems, Measurement, and Control*, 108(3):163–171, 1986. doi: 10.1115/1.3143764.
- [7] Stefano Chiaverini, Bruno Siciliano, and Olav Egeland. Review of the damped least-squares inverse kinematics with experiments on an industrial robot manipulator. *IEEE Transactions on Control Systems Technology*, 2(2):123–134, 1994. doi: 10.1109/87.294335.
- [8] Patrick Beeson and Barrett Ames. TRAC-IK: An open-source library for improved solving of generic inverse kinematics. In *2015 IEEE-RAS 15th International Conference on Humanoid Robots*, pages 928–935, 2015. doi: 10.1109/HUMANOIDS.2015.7363472.
- [9] Bruce Wingo, Ajay Suresha Sathya, St phane Caron, Seth Hutchinson, and Justin Carpentier. Linear-time differential inverse kinematics: An augmented lagrangian perspective. In *Robotics: Science and Systems XX*, 2024. doi: 10.15607/RSS.2024.XX.110.
- [10] Signe Moe, Gianluca Antonelli, Andrew R. Teel, Kristin Y. Pettersen, and Johannes Schrimpf. Set-based tasks within the singularity-robust multiple task-priority inverse kinematics framework: General formulation, stability analysis, and experimental results. *Frontiers in Robotics and AI*, 3:16, 2016. doi: 10.3389/frobt.2016.00016.

- [11] Dmitry Berenson, Siddhartha Srinivasa, and James Kuffner. Task space regions: A framework for pose-constrained manipulation planning. *The International Journal of Robotics Research*, 30(12):1435–1460, 2011. doi: 10.1177/0278364910396389.
- [12] John R. Rice. The algorithm selection problem. In Morris Rubinoff and Marshall C. Yovits, editors, *Advances in Computers*, volume 15, pages 65–118. Academic Press, 1976. doi: 10.1016/S0065-2458(08)60520-3.
- [13] Bernd Bischl, Pascal Kerschke, Lars Kotthoff, Marius Lindauer, Yuri Malitsky, Alexandre Fréchet, Holger Hoos, Frank Hutter, Kevin Leyton-Brown, Kevin Tierney, and Joaquin Vanschoren. ASlib: A benchmark library for algorithm selection. *Artificial Intelligence*, 237:41–58, 2016. doi: 10.1016/j.artint.2016.04.003.
- [14] Xinyi Yuan, Weiwei Wan, and Kensuke Harada. IKSel: Selecting good seed joint values for fast numerical inverse kinematics iterations. *IEEE Transactions on Automation Science and Engineering*, 23:4410–4427, 2026. doi: 10.1109/TASE.2026.3659225.
- [15] Jacinto Colan, Ana Davila, and Yasuhisa Hasegawa. Variable step sizes for iterative jacobian-based inverse kinematics of robotic manipulators. *IEEE Access*, 12:87909–87922, 2024. doi: 10.1109/ACCESS.2024.3418206.
- [16] Chung Min Kim, Brent Yi, Hong Suk Choi, Yi Ma, Ken Goldberg, and Angjoo Kanazawa. PyRoki: A modular toolkit for robot kinematic optimization. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1312–1319, 2025. doi: 10.1109/IROS60139.2025.11246651.
- [17] Pablo C. Lopez-Custodio, Yuhe Gong, and Luis F. C. Figueredo. GeoFIK: A fast and reliable geometric solver for the IK of the Franka arm based on screw theory enabling multiple redundancy parameters, 2025. URL <https://arxiv.org/abs/2503.03992>. Preprint; revised in 2026; ICRA 2026 acceptance reported by the authors; final proceedings metadata unverified.
- [18] Cael Yasutake, Andrew H. Liu, Zachary Kingston, and Brian Plancher. HJCD-IK: Gpu-accelerated inverse kinematics through batched hybrid jacobian coordinate descent, 2025. URL <https://arxiv.org/abs/2510.07514>. Preprint; first posted in 2025, revised in 2026; accepted to the 2026 IEEE/RSJ International Conference on Intelligent Robots and Systems.
- [19] Liangting Wu and Roberto Tron. IKSPARK: Obstacle-aware inverse kinematics via convex optimization, 2024. URL <https://arxiv.org/abs/2403.12235>. Preprint; first posted in 2024, latest revision in 2026.
- [20] Hannes Gamper, Laura Rodrigo Pérez, Andreas Mueller, Alejandro Díaz Rosales, and Mario Di Castro. An inverse kinematics algorithm with smooth task switching for redundant robots. *IEEE Robotics and Automation Letters*, 9(5):4527–4534, 2024. doi: 10.1109/LRA.2024.3379860.
- [21] Christoph Schuetz, Thomas Buschmann, Joerg Baur, Julian Pfaff, and Heinz Ulbrich. Predictive online inverse kinematics for redundant manipulators. In *2014 IEEE International Conference on Robotics and Automation*, pages 5056–5061, 2014. doi: 10.1109/ICRA.2014.6907600.
- [22] Łukasz Woliński and Marek Wojtyra. An inverse kinematics solution with trajectory scaling for redundant manipulators. *Mechanism and Machine Theory*, 191:105493, 2024. doi: 10.1016/j.mechmachtheory.2023.105493.
- [23] Łukasz Woliński and Marek Wojtyra. Predictive inverse kinematics with trajectory scaling for redundant manipulators based on quadratic optimization. *Mechanism and Machine Theory*, 209:105988, 2025. doi: 10.1016/j.mechmachtheory.2025.105988.
- [24] Vamsi Krishna Origanti, Adrian Danzglock, and Frank Kirchner. Look ahead optimization for managing nullspace in cartesian impedance control of dual-arm robots. In *2025 IEEE/SICE International Symposium on System Integration*, pages 990–997, 2025. doi: 10.1109/SII59315.2025.10871056.
- [25] Yachen Zhang and Ryo Kikuuwe. Ensuring viability: A QP-based inverse kinematics for handling joint range, velocity and acceleration limits, as well as whole-body collision avoidance. *Journal of Intelligent & Robotic Systems*, 112(1):16, 2026. doi: 10.1007/s10846-025-02335-z.
- [26] Ivan Boschi, Alessandro De Toni, Roberto Di Leva, Edoardo Idà, and Marco Carricato. Handling transitions across singularities for UR-like serial robots. *IEEE Robotics and Automation Letters*, 11(3):2714–2721, 2026. doi: 10.1109/LRA.2026.3653295.
- [27] Jeremy Morgan, David Millard, and Gaurav S. Sukhatme. CppFlow: Generative inverse kinematics for efficient and robust cartesian path planning. In *2024 IEEE International Conference on Robotics and Automation*, 2024. doi: 10.1109/ICRA57147.2024.10611724.

- [28] Yucheng Tang, Xi Huang, Yongzhou Zhang, Tao Chen, Ilshat Mamaev, and Björn Hein. ETA-IK: Execution-time-aware inverse kinematics for dual-arm systems. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 7504–7511, 2025. doi: 10.1109/IROS60139.2025.11247583.
- [29] Barrett Ames, Jeremy Morgan, and George Konidaris. IKFlow: Generating diverse inverse kinematics solutions. *IEEE Robotics and Automation Letters*, 7(3):7177–7184, 2022. doi: 10.1109/LRA.2022.3181374.
- [30] Jan-Gerrit Habekost, Erik Strahl, Philipp Allgeuer, Matthias Kerzel, and Stefan Wermter. CycleIK: Neuro-inspired inverse kinematics. In *Artificial Neural Networks and Machine Learning – ICANN 2023*, volume 14254 of *Lecture Notes in Computer Science*, pages 457–470. Springer, 2023. doi: 10.1007/978-3-031-44207-0_38.
- [31] Oliver Limoyo, Filip Marić, Matthew Giamou, Petra Alexson, Ivan Petrović, and Jonathan Kelly. Generative graphical inverse kinematics. *IEEE Transactions on Robotics*, 41:1002–1018, 2025. doi: 10.1109/TRO.2024.3521862.
- [32] Jacket Demby’s, Ramy Farag, and Guilherme N. DeSouza. Inverse kinematics of robotic manipulators using a new learning-by-example method. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 9534–9541, 2024. doi: 10.1109/IROS58592.2024.10802048.
- [33] Duc Tien Nguyen, Van Thanh Tri Nguyen, Truong Do, and Vu Linh Nguyen. Multi-solution inverse kinematics for robotic manipulators via permutation-invariant set prediction. *IEEE Robotics and Automation Letters*, 11(8):9803–9810, 2026. doi: 10.1109/LRA.2026.3703280.
- [34] Jiahao Yang, Shenhao Yan, Fan Feng, Chengsi Yao, Ge Wang, Zhixin Mai, Yiming Zhao, and Yatong Han. MimicIK: Real-time generative inverse kinematics from teleoperation with fk consistency, 2026. URL <https://arxiv.org/abs/2606.15148>. Preprint.
- [35] Meenalochani Jayabalan, Karunamoorthy Loganathan, and Palanikumar Kayaroganam. A novel hybrid ik architecture for robotic arms: Iterative refinement of soft-computing approximations with validation on abb irb-1200 robotic arm. *Machines*, 14(3):292, 2026. doi: 10.3390/machines14030292.
- [36] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- [37] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.
- [38] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011. URL <https://www.jmlr.org/papers/v12/pedregosa11a.html>.